



Plan de pruebas del backend de la aplicación SoGaffer

Los niveles de prueba que se van a aplicar son los siguientes:

- **Pruebas unitarias.** Se utilizará la técnica de prueba de métodos, usando técnicas de caja negra (partición equivalente y AVL) para la definición de los casos de prueba de cada método de cada clase o componente. Será necesaria la utilización de Junit y Mockito para las tres capas, además de WireMock para aquellos servicios que llamen a la API de Sorare, y WebMvcTest para la capa de presentación.
- **Pruebas de integración.** La estrategia para la definición del orden de las pruebas de integración será jerárquica. Se probará:
 - La integración entre la capa de negocio y la de persistencia. En este caso, para la definición de los casos de prueba se utilizarán técnica de métodos y caja negra y se utilizará JUnit.
 - La integración entre las tres capas (presentación -controlador-, negocio -servicios- y persistencia -repositorios-). En este caso, para la definición de los casos de prueba se utilizarán técnica de casos de uso y se utilizará Junit.

A continuación, se muestra una especificación detallada de los casos de prueba a aplicar en cada nivel mencionado anteriormente.

PRUEBAS UNITARIAS

- Pruebas unitarias de la capa de persistencia

No se llevarán a cabo pruebas unitarias de la capa de persistencia:

- Dado que las interfaces IRefreshTokenRepository e IUserRepository heredan de JpaRepository y sus métodos son generados automáticamente por Spring Data JPA, no contienen lógica propia que testear.
- Aunque ISorareTokenRepository define dos consultas personalizadas mediante la anotación @Query, requiere una base de datos real para verificar su comportamiento, por lo que sus pruebas se llevarán a cabo en la capa de integración.

- Pruebas unitarias de la capa de negocio

Para poder llevar a cabo estas pruebas, será necesario el uso de mocks para las interfaces IEncryptionService, IJwtTokenService, IRefreshTokenService, ISorareGraphQLService, ISorareLoginService e ISorareTokenService. Se aplica



prueba de métodos, siendo los casos de prueba definidos para cada método los siguientes:

Interfaz IEncryptionService

- encrypt (plainText: String)

Identificador	Entrada	Valor esperado
UIES.1a	texto_plano	Texto cifrado (y distinto al plano)

- decrypt (cipheredText)

Identificador	Entrada	Valor esperado
UIES.1b	Texto cifrado	texto_plano (y distinto al cifrado)

Interfaz IJwtTokenService

- generateToken (username: String)

Identificador	Entrada	Valor esperado
UIJTS.1a	correct@email.com	JWT firmado con la clave privada de 32 bytes y los siguientes claims: - subject = correct@email.com - issuedAt = ahora - expiration = ahora + 30 días - issuer = SoGaffer

- getSigningKey ()

Identificador	Entrada	Valor esperado
UIJTS.1b	/	Clave secreta con algoritmo HmacSHA256

Interfaz IRefreshTokenService

- generateRefreshToken (user: UserDto)

Identificador	Entrada	Valor esperado
UIRTS.1a	user = (id = 1, sorareId = 00000000-0000-0000-0000-000000000000, email = test@test.com, updatedAt = null)	Token de refresco de 32 bytes
UIRTS.1b	user = null	DataIntegrityViolationException



UIRTS.1c	user = (id = 1, sorareId = 00000000-0000-0000-0000-000000000000, email = test@test.com, updatedAt = null)	(el usuario no puede ser nulo) Excepción (no se pudo persistir el token de refresco)
----------	---	---

Interfaz ISorareGraphQLService

- signIn (loginDto: LoginDto)

Identificador	Entrada	Valor esperado
UISGQLS.1a	loginDto = (email = correct@email.com, password = hashed_password)	(currentUser = (sorareId = 00000000-0000-0000-0000-000000000000, email = test@test.com), jwtToken= (token = sorare_token, expiredAt = ahora + 30 días), errors = [])
UISGQLS.1b	loginDto = (email = correct@email.com, password = incorrect_hashed_password)	IllegalStateException (no se pudo leer el archivo que contiene la mutación del login)
UISGQLS.1c	loginDto = (email = correct@email.com, password = hashed_password)	InvalidCredentialsException (contraseña incorrecta)
UISGQLS.1d	loginDto = (email = correct@email.com, password = hashed_password)	Excepción (el servicio de Sorare no está disponible)

Interfaz ISorareTokenService

- invalidateActiveTokens (userId: int): void

Identificador	Entrada	Valor esperado
UISTS.1a	1	- Todos los tokens pasan a valid=false - Se llama a saveAll()
UISTS.1b	1	El servicio de persistencia no está disponible

- persistSorareToken (user: UserDto, jwtTokenDto: SorareJwtTokenDto)

Identificador	Entrada	Valor esperado
UISTS.1c	user = (id = 1, sorareId =	- Se llama a save una vez



	00000000-0000-0000-0000-0000000000000000, email = test@test.com, updatedAt = null), jwtTokenDto = (token = sorare_token, expiredAt = ahora + 1 semana)	- Los tokens quedan invalidados (active := false)
UISTS.1d	user = null, jwtTokenDto = (token = sorare_token, expiredAt = ahora + 1 semana)	NullPointerException (no el usuario no puede ser nulo)
UISTS.1e	user = (id = 1, sorareId = 00000000-0000-0000-0000-0000000000000000, email = test@test.com, updatedAt = null), jwtTokenDto = null	NullPointerException (no el token no puede ser nulo)
UISTS.1f	user = (id = 1, sorareId = 00000000-0000-0000-0000-0000000000000000, email = test@test.com, updatedAt = null), jwtTokenDto = (token = sorare_token, expiredAt = ahora + 1 semana)	Excepción (el servicio de persistencia no está disponible)

- Pruebas unitarias de la capa de presentación

Para poder llevar a cabo estas pruebas, se utilizará la anotación @WebMvcTest, de forma que se levante únicamente el contexto web. Seguirá siendo necesario el uso de mocks para la interfaz IAuthService.

Se aplica prueba de métodos, siendo los casos de prueba definidos para cada método los siguientes:

- login (loginDto: LoginDto, response: HttpServletResponse)

Identificador	Entrada	Valor esperado
UAC.1a	(correct@email.com, correct_password), sogaffer.cookie.secure = false	- 200 OK - (accessToken=token, expiresIn=3600) - Cookie refresh_token contiene valor correcto, HttpOnly = true y Secure = false
UAC.1b	(correct@email.com,	- 200: OK



	correct_password), sogaffer.cookie.secure = true	- (accessToken=token, expiresIn=3600) - Cookie refresh_token contiene valor correcto, HttpOnly = true y Secure = true
UAC.1c	(null, correctpassword)	400: Bad Request
UAC.1d	(blank, correctpassword)	400: Bad Request
UAC.1e	(correct@email.com, null)	400: Bad Request
UAC.1f	(correct@email.com, blank)	400: Bad Request
UAC.1g	JSON inválido	400: Bad Request
UAC.1h	(correct@email.com, incorrect_password), InvalidCredentialsException	401: Unauthorized
UAC.1i	(correct@email.com, correct_password), Exception	500: Internal Server Error

PRUEBAS DE INTEGRACIÓN

El orden de las pruebas y los casos de prueba a realizar se realizará siguiendo una estrategia *bottom-up*, donde se probarán primero las capas más internas de la API (empezando por los repositorios) y se ascenderá progresivamente hasta el controlador, integrando en cada capa aquellas que estén por debajo del nivel en el que se encuentren.

Dado que las pruebas unitarias ya verifican el comportamiento aislado de cada componente, las pruebas de integración no necesitan un mock de las capas inferiores, al haber sido estas validadas individualmente. Por tanto, en cada nivel de integración se pueden usar las implementaciones reales de las capas inferiores sustituyendo únicamente las dependencias externas al sistema.

La base de datos se reemplazará con una embebida WireMock en memoria mediante H2 que ejecutará las mismas *queries* JPA sobre un esquema equivalente; la API externa de Sorare, por su parte, se simulará con WireMock).

Héctor González Iglesias